BiblioFlow — Change request CR-BF-01

Specifica dei casi di test pre-modifica

Ingegneria, Gestione ed Evoluzione del Software — A.A. 2025/2026

Gruppo

Mario Celzo — NF22500193

Renato Mancino — NF22500242

Alfonso Maria Ferrara — NF22500210

Repository: https://github.com/mariocelzo/biblioflow-app — codice verificato al commit `066f198`

Data: 13 ottobre 2026

Indice

Specifica dei casi di test pre-modifica — automazioni, notifiche e SSE 3
Scopo 3
Convenzione degli identificativi 3
Ambiente e dati 3
Casi di test 3
Tracciabilità verso i test eseguibili 4
Esecuzione 4
Specifica dei casi di test pre-modifica — area prenotazioni 5
Scopo e baseline 5
Dati e convenzioni comuni 5
Matrice di copertura 5
Casi di test 6
Osservazioni per le fasi successive 8
Specifica dei casi di test pre-modifica — admin e UI 8
Scopo e tracciabilità 8
Matrice ruoli-operazioni verificata 8
Matrice dei casi 9
Specifica dettagliata 9
Note per i test post-modifica 11

Specifica dei casi di test pre-modifica — automazioni, notifiche e SSE

Scopo

Questa specifica fotografa il comportamento originale dell'area assegnata a Renato prima degli interventi di Fase 4 della CR-BF-01. I casi verificano automazioni, notifiche e il contratto del canale Server-Sent Events dei posti.

I casi non dimostrano ancora criteri di accettazione della change request: costituiscono la baseline di regressione da mantenere o aggiornare in modo esplicito nelle fasi successive.

Convenzione degli identificativi

Gli identificativi sono stabili e divisi per area per evitare collisioni con le suite degli altri componenti:

- `PRE-AUT-nnn`: automazioni periodiche;
- `PRE-NOT-nnn`: notifiche persistenti e relativa API;
- `PRE-SSE-nnn`: contratto realtime SSE.

Ogni ID compare sia in questo documento sia nel nome del corrispondente test Vitest.

Ambiente e dati

- PostgreSQL 16 reale, isolato tramite il servizio `postgres-test` di Docker Compose;
- URL verificato prima di ogni sincronizzazione: il nome del database deve identificarlo esplicitamente come database di test;
- schema Prisma corrente applicato con `prisma db push` esclusivamente sul DB di test, anche quando la CI ha inizialmente applicato le sole migrazioni versionate;
- database ripulito prima di ogni caso di integrazione;
- date e orari costruiti rispetto all'istante di esecuzione per rendere riproducibili promemoria e no-show;
- test SSE eseguiti in ambiente Node, senza browser o connessioni di rete.

Casi di test

Automazioni

ID	Scenario e precondizioni	Azione	Risultato atteso sulla baseline
`PRE-AUT-001`	Prenotazione `CONFERMATA` con inizio tra 15 e 20 minuti e nessun promemoria già inviato.	Eseguire `sendCheckInReminders()`.	Una notifica `CHECK_IN_REMINDER` non letta, con link alla prenotazione e label `Fai check-in`; viene creato anche un log `AUTOMATION`.
`PRE-AUT-002`	Stesse condizioni di `PRE-AUT-001`.	Eseguire due volte il promemoria nello stesso giorno.	La prima esecuzione invia una notifica, la seconda zero; nel DB rimane un solo promemoria.
`PRE-AUT-003`	Prenotazione `CONFERMATA` iniziata da oltre 15 minuti, senza check-in, con posto occupato.	Eseguire `releaseNoShowReservations()`.	Stato `NO_SHOW`, posto `DISPONIBILE`, notifica `ALERT` e log `NO_SHOW_AUTO` con riferimenti a prenotazione,

ID	Scenario e precondizioni	Azione	Risultato atteso sulla baseline
			utente e posto.
PRE-AUT-004	Prenotazione già in stato SCADUTA e con orario passato.	Eseguire `releaseNoShowReservations()`.	Nessuna prenotazione elaborata: SCADUTA resta invariato e non vengono creati notifiche o log. Il sistema originale non produce automaticamente questo stato.

Notifiche

ID	Scenario e precondizioni	Azione	Risultato atteso sulla baseline
PRE-NOT-001	Un utente possiede una prenotazione recente nella stessa sala del posto appena liberato.	Eseguire `notifyAvailableSeat()`.	L'utente riceve una notifica `INFO`, non letta, con action URL `/prenota`.
PRE-NOT-002	L'utente possiede una notifica letta e una non letta.	Chiamare `GET /api/notifiche` con `letta=false`, `limit=1` e `offset=0`.	HTTP 200; risposta con `success`, lista filtrata, `count=1`, `totale=2` e `nonLette=1`.

Canale SSE dei posti

ID	Scenario e precondizioni	Azione	Risultato atteso sulla baseline
PRE-SSE-001	Nessuna precondizione applicativa.	Chiamare `GET /api/sse/posti` e leggere il primo chunk.	HTTP 200, content type `text/event-stream`, cache disabilitata, connessione keep-alive, buffering nginx disabilitato e commento iniziale `: connected`.
PRE-SSE-002	Client sui canali `posti`, `user-1` e wildcard `*`.	Emettere `posto-update` sul canale `posti`.	Ricevono l'evento soltanto `posti` e wildcard; il formato è `event: ...`, seguito da `data: <JSON>` e doppio newline.
PRE-SSE-003	Emitter disponibile.	Chiamare `emitPostoUpdate()` con tutti i campi.	Evento `posto-update` sul canale `posti` con `postoId`, `stato`, `numero`, `salaId`, `salaNome` e timestamp ISO.
PRE-SSE-004	Stream aperto sul canale `posti`.	Cancellare il reader dello stream.	Il client viene rimosso dal registro dell'emitter e il conteggio torna al valore iniziale.

Tracciabilità verso i test eseguibili

Area	File Vitest
Automazioni e notifiche	`tests/pre-modifica/automation-notifications.test.ts`
SSE posti	`tests/pre-modifica/sse-posti.test.ts`

Esecuzione

```
npm run test:db:prepare
npm test
npm run test:db:down
```

La preparazione rifiuta URL che non identificano esplicitamente un database di test. Il database è effimero e viene rimosso al termine della verifica.

Specifica dei casi di test pre-modifica — area prenotazioni

Scopo e baseline

Questo documento descrive il comportamento dell'area prenotazioni prima delle modifiche di CR-BF-01. È il primo deliverable di testing richiesto dal corso e non dimostra ancora alcun criterio di accettazione della change request.

- Task: BIB-19 — Specifica casi di test pre-modifica - area prenotazioni
- Baseline funzionale: tag Git `baseline-pre-cr-bf-01`
- Riferimento di esecuzione: `main` al commit `7469969`, che aggiunge l'infrastruttura di test senza modificare i flussi qui caratterizzati
- Implementazione eseguibile: `tests/pre-modifica/prenotazioni.test.ts`
- Comando: `npm test`

I test invocano direttamente i route handler Next.js e isolano l'accesso a Prisma con mock. In questo modo fissano il contratto applicativo della baseline, sono deterministici e non condividono dati con le altre suite pre-modifica.

Dati e convenzioni comuni

- Utente studente: `pre-user-1`
- Posto: `pre-posto-a1`, numero `A1`, stato `DISPONIBILE`
- Sala: aperta dalle `08:00` alle `18:00`
- Data ordinaria: `2030-01-15`
- Stato iniziale della prenotazione, salvo diversa indicazione: `CONFERMATA`
- Una risposta non deve produrre scritture quando il caso è rifiutato.

Le date `@db.Date` sono rappresentate da Prisma con la data effettiva, mentre gli orari `@db.Time` hanno data convenzionale `1970-01-01`. I casi TC-PRE-007 e TC-PRE-010 conservano esplicitamente questa rappresentazione perché fanno emergere due anomalie della baseline.

Matrice di copertura

ID	Funzione	Tipo	Esito baseline
TC-PRE-001	Creazione	Positivo	Prenotazione confermata, log e notifica creati
TC-PRE-002	Creazione / validazione	Negativo	Campi obbligatori mancanti rifiutati
TC-PRE-003	Validazione orari	Negativo	Intervallo fuori apertura rifiutato
TC-PRE-004	Validazione orari	Caratterizzazione difetto	Intervallo con fine precedente all'inizio accettato
TC-PRE-005	Creazione / sovrapposizione	Negativo	Posto già occupato nello slot rifiutato
TC-PRE-006	Cancellazione	Positivo	Stato aggiornato e log creato
TC-PRE-007	Check-in dedicato	Caratterizzazione difetto	Check-in valido sulla data della prenotazione considerato

ID	Funzione	Tipo	Esito baseline
			scaduto
TC-PRE-008	Check-in legacy	Positivo	Stato prenotazione e posto aggiornati
TC-PRE-009	Estensione	Negativo	Estensione sovrapposta rifiutata
TC-PRE-010	Estensione	Caratterizzazione difetto	Estensione libera rifiutata per durata calcolata tra date incompatibili

Casi di test

TC-PRE-001 — Creazione valida

Precondizioni. L'utente, il posto e la sala esistono. L'utente e il posto non hanno prenotazioni attive sovrapposte.

Passi.

1. Inviare `POST /api/prenotazioni` con utente e posto validi, data `2030-01-15`, intervallo `09:00–11:00`.
2. Ispezionare risposta e scritture richieste al livello di persistenza.

Risultato atteso. HTTP 201; prenotazione in stato `CONFERMATA`; creazione di un evento `PRENOTAZIONE_CREATA` e di una notifica `PRENOTAZIONE`.

TC-PRE-002 — Campo obbligatorio mancante

Precondizioni. Nessuna.

Passi.

1. Inviare `POST /api/prenotazioni` senza `oraFine`.

Risultato atteso. HTTP 400 con indicazione dei campi obbligatori; nessuna prenotazione creata.

TC-PRE-003 — Intervallo fuori apertura

Precondizioni. Sala aperta `08:00–18:00`; utente e posto esistenti e liberi.

Passi.

1. Inviare `POST /api/prenotazioni` per `07:30–09:00`.

Risultato atteso. HTTP 400 con gli orari della sala; nessuna prenotazione creata.

TC-PRE-004 — Intervallo invertito

Precondizioni. Utente e posto esistenti e liberi.

Passi.

1. Inviare `POST /api/prenotazioni` per `11:00–09:00`.

Risultato atteso sulla baseline. HTTP 201. Il sistema non verifica che `oraFine` sia successiva a `oraInizio`. Il caso è una caratterizzazione di un difetto, non il comportamento desiderato dalla CR.

TC-PRE-005 — Sovrapposizione sul posto

Precondizioni. Il posto A1 ha una prenotazione attiva `10:00–12:00`; il nuovo utente non ha sovrapposizioni personali.

Passi.

1. Inviare `POST /api/prenotazioni` sullo stesso posto per `09:00–11:00`.

Risultato atteso. HTTP 409 con indicazione che il posto è già prenotato; nessuna nuova prenotazione.

TC-PRE-006 — Cancellazione logica

Precondizioni. Prenotazione esistente in stato `CONFERMATA`.

Passi.

1. Inviare `PATCH /api/prenotazioni/{id}` con azione `cancella`.

Risultato atteso. HTTP 200; stato aggiornato a `CANCELLATA`; evento `PRENOTAZIONE_CANCELLATA` associato alla prenotazione.

TC-PRE-007 — Check-in mediante endpoint dedicato

Precondizioni. Sessione dell'intestatario; prenotazione `CONFERMATA` il `2030-01-15` alle `09:00`; timestamp richiesto `08:50`, dentro la finestra dichiarata di quindici minuti.

Passi.

1. Inviare `POST /api/prenotazioni/{id}/check-in` con timestamp `2030-01-15T08:50:00.000Z`.

Risultato atteso sulla baseline. HTTP 400, "Il periodo di check-in è scaduto". L'endpoint confronta il timestamp completo con un campo `@db.Time` ancorato al 1970. Nessun aggiornamento. È un difetto caratterizzato.

TC-PRE-008 — Check-in mediante azione legacy

Precondizioni. Prenotazione esistente in stato `CONFERMATA`.

Passi.

1. Inviare `PATCH /api/prenotazioni/{id}` con azione `check-in`.

Risultato atteso. HTTP 200; prenotazione in `CHECK_IN`; posto in `OCCUPATO`; evento `CHECK_IN` creato.

TC-PRE-009 — Estensione in conflitto

Precondizioni. Prenotazione attiva `09:00–11:00`; altra prenotazione attiva sullo stesso posto nello slot richiesto.

Passi.

1. Inviare `POST /api/prenotazioni/{id}/estendi` con `nuovaOraFine: 13:00`.

Risultato atteso. HTTP 409 con indicazione di indisponibilità; orario originale invariato.

TC-PRE-010 — Estensione libera con rappresentazione Prisma reale

Precondizioni. Prenotazione attiva del `2030-01-15`, orari Prisma `09:00–11:00` ancorati al `1970-01-01`; nessuna sovrapposizione.

Passi.

1. Inviare `POST /api/prenotazioni/{id}/estendi` con `nuovaOraFine: 13:00`.

Risultato atteso sulla baseline. HTTP 400 per durata superiore a otto ore; nessun aggiornamento. La durata usa la data reale per la nuova fine e il 1970 per l'inizio. È un difetto caratterizzato.

Osservazioni per le fasi successive

La baseline presenta due percorsi di check-in con contratti differenti. L'endpoint dedicato applica sessione, ownership e finestra temporale ma soffre il confronto tra data e `@db.Time`; la `PATCH` legacy completa il check-in senza tali controlli. Anche l'estensione combina data reale e orari ancorati al 1970. Queste evidenze dovranno essere riprese nell'impact analysis e confrontate con i casi post-modifica, senza correggerle durante la Fase 1.

Specifica dei casi di test pre-modifica — admin e UI

Scopo e tracciabilità

Questa sezione fotografa il comportamento che l'area amministrativa e il flusso di prenotazione UI devono conservare durante CR-BF-01. I casi costituiscono la baseline di regressione per **CA-06**.

- Task: BIB-21 — Specifica casi di test pre-modifica - admin e UI
- Responsabile operativo: Renato Mancino, in sostituzione anticipata dell'assegnazione originaria ad Alfonso
- Baseline funzionale: tag Git `baseline-pre-cr-bf-01`
- Riferimento di esecuzione: `main` al commit `7469969`
- Implementazioni eseguibili:
  - `tests/pre-modifica/login.test.ts`
  - `tests/pre-modifica/admin-ui.test.ts`
- Comando: `npm test`

La numerazione parte da `TC-PRE-011` per non collidere con i casi prenotazioni `TC-PRE-001`–`TC-PRE-010` sviluppati nella PR separata della task BIB-19.

Matrice ruoli-operazioni verificata

Operazione	Anonimo	Studente	Bibliotecario	Admin
Pagina `/admin/*`	Redirect login	Accesso negato dalle verifiche di ruolo applicative	Consentito	Consentito
API admin	401	403	Consentita, salvo operazioni riservate	Consentita
Attivazione/ disattivazione utente	401	403	403	Consentita
Dettagli e storico utente	401	403	Consentita	Consentita
Gestione prestiti/prenotazioni/statistiche	401	403	Consentita	Consentita
Pagina `/prenota`	Redirect login	Consentita con sessione	Consentita con sessione	Consentita con sessione

Matrice dei casi

ID	Area	Scenario	Risultato atteso baseline
TC-PRE-011	Login	Credenziali corrette, account attivo	Utente restituito e ultimo accesso aggiornato
TC-PRE-012	Login	Password errata	Errore credenziali, nessun aggiornamento
TC-PRE-013	Login	Account disattivato	Errore dedicato prima del confronto password
TC-PRE-014	Middleware UI	Pagina admin senza sessione	Redirect a login con callback URL
TC-PRE-015	Middleware API	API admin senza sessione	HTTP 401
TC-PRE-016	Middleware UI	Pagina prenota con cookie sessione	Richiesta lasciata proseguire
TC-PRE-017	Statistiche	Studente autenticato	HTTP 403 senza query dati
TC-PRE-018	Utenti	Bibliotecario modifica stato account	HTTP 403; operazione riservata ad Admin
TC-PRE-019	Utenti	Bibliotecario legge dettagli	Dati utente e conteggi restituiti
TC-PRE-020	Prestiti	Admin registra restituzione	Stato restituito, log e notifica
TC-PRE-021	Prenotazioni admin	Bibliotecario annulla prenotazione in check-in	Cancellazione e posto liberato
TC-PRE-022	Statistiche	Distribuzione no-show	Valori e percentuali aggregati
TC-PRE-023	UI prenota	Contratto sorgente del payload	Endpoint e campi della richiesta invariati
TC-PRE-024	UI prenota	Invio payload valido	HTTP 201 e prenotazione confermata

Specifica dettagliata

TC-PRE-011 — Login valido

Precondizioni. Account studente attivo, configurato con password; confronto hash positivo.

Passi. Autenticarsi mediante provider Credentials con email e password corrette.

Risultato atteso. Il provider restituisce i dati utente senza hash e aggiorna `ultimoAccesso`.

TC-PRE-012 — Password errata

Precondizioni. Account attivo esistente; confronto hash negativo.

Passi. Autenticarsi con password errata.

Risultato atteso. Errore “Credenziali non valide”; `ultimoAccesso` invariato.

TC-PRE-013 — Account disattivato

Precondizioni. Account esistente con `attivo=false`.

Passi. Tentare il login.

Risultato atteso. Errore “Account disabilitato. Contatta la biblioteca.”; la password non viene confrontata.

TC-PRE-014 — Pagina admin anonima

Precondizioni. Cookie di sessione assente.

Passi. Richiedere `/admin/prenotazioni`.

Risultato atteso. Redirect temporaneo a `/login?callbackUrl=/admin/prenotazioni`.

TC-PRE-015 — API admin anonima

Precondizioni. Cookie di sessione assente.

Passi. Richiedere `/api/admin/statistiche`.

Risultato atteso. HTTP 401 con errore `Non autenticato`.

TC-PRE-016 — UI prenotazione autenticata

Precondizioni. Cookie `authjs.session-token` presente.

Passi. Richiedere `/prenota`.

Risultato atteso. Il middleware lascia proseguire la richiesta.

TC-PRE-017 — Statistiche richieste da studente

Precondizioni. Sessione con ruolo `STUDENTE`.

Passi. Richiedere il tasso no-show all'API statistiche admin.

Risultato atteso. HTTP 403; nessuna query di aggregazione.

TC-PRE-018 — Modifica utente richiesta da bibliotecario

Precondizioni. Sessione con ruolo `BIBLIOTECARIO`.

Passi. Tentare di disattivare un utente tramite `PATCH /api/admin/utenti/{id}`.

Risultato atteso. HTTP 403; nessun aggiornamento. La baseline riserva l'operazione ad `ADMIN`.

TC-PRE-019 — Dettagli utente per bibliotecario

Precondizioni. Sessione bibliotecario; utente esistente con prenotazioni, prestiti e un no-show.

Passi. Richiedere `GET /api/admin/utenti/{id}`.

Risultato atteso. HTTP 200 con profilo, storico e conteggi di prenotazioni, prestiti e no-show.

TC-PRE-020 — Restituzione prestito

Precondizioni. Sessione admin; prestito attivo non scaduto.

Passi. Inviare l'azione `RESTITUISCI` all'API prestiti admin.

Risultato atteso. Stato `RESTITUITO`, data restituzione valorizzata, log e notifica; zero giorni di ritardo.

TC-PRE-021 — Cancellazione prenotazione admin

Precondizioni. Sessione bibliotecario; prenotazione in `CHECK_IN` con posto `OCCUPATO`.

Passi. Inviare l'azione `ANNULLA_SINGOLA` all'API prenotazioni admin.

Risultato atteso. Prenotazione `CANCELLATA`, posto `DISPONIBILE`, log e notifica.

TC-PRE-022 — Statistica tasso no-show

Precondizioni. Sessione staff; aggregato con 2 no-show, 6 completate e 2 cancellate.

Passi. Richiedere `tipo=tasso-noshow`.

Risultato atteso. Distribuzione 20% no-show, 60% completate, 20% altre.

TC-PRE-023 — Contratto UI/API prenotazione

Precondizioni. Sorgente baseline della pagina `/prenota`.

Passi. Verificare in modo eseguibile endpoint e campi serializzati dal flusso di conferma.

Risultato atteso. `POST /api/prenotazioni` con `userId`, `postoId`, data, orari e margine pendolare. Il passaggio di `userId` dal client è comportamento della baseline, non il comportamento sicuro richiesto dalla CR.

TC-PRE-024 — Conferma prenotazione dalla UI

Precondizioni. Utente, sala e posto esistenti; nessuna sovrapposizione.

Passi. Inviare all'API il payload prodotto dalla pagina `/prenota`.

Risultato atteso. HTTP 201 e prenotazione `CONFERMATA`.

Note per i test post-modifica

I casi di login, gestione utenti, prestiti, statistiche e navigazione protetta costituiscono regressione invariata per CA-06. I casi TC-PRE-021, TC-PRE-023 e TC-PRE-024 intersecano invece i ripple effect della CR: la cancellazione admin dovrà attivare la promozione dalla lista d'attesa e l'identità della prenotazione non dovrà più provenire dal payload client. Nella specifica post-modifica dovranno quindi essere classificati come modificati, mantenendo esplicito il confronto con questa baseline.